Syllabus Extraction & Verification Studio

Comprehensive Technical Methodology, Architectural Specifications & System Review

Target Project: Academic Syllabus Extractor & Verification Studio **Architecture:** Deterministic Hybrid (Heuristic + Grounded LLM + OCR) **Status:** Production Ready (27/27 Test Suite Passed)

1. Executive Summary & Core Objectives

The **Syllabus Extraction & Verification Studio** is an enterprise-grade academic data intelligence system designed to ingest heterogeneous university curriculum documents (PDFs and PPTX presentations), parse complex multi-column structures, extract fine-grained academic syllabus specifications, enforce 100% substring grounding against original source texts, and seamlessly map syllabus topics to reference textbooks.

Unlike naive LLM ingestion systems that suffer from hallucination and token drift, this system employs a **deterministic, hybrid architecture**. Language models are utilized strictly for *boundary disambiguation* (identifying line ranges), while all data extraction, text parsing, Bloom's taxonomy mapping, and referential integrity checks are governed by deterministic Python parsing algorithms and a mathematical Verification Layer.

> **Core Engineering Guarantee:** Every extracted entity (course code, unit title, verbatim text, topic, course outcome, and prescribed book) is rigorously grounded by substring verification against the original document text. Hallucinated items are mathematically impossible to emit.

2. Complete Technology Stack & APIs

A. External APIs & Cloud Services

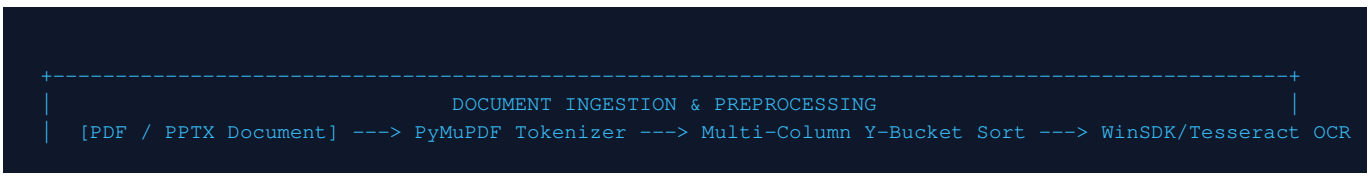
Service / API	Target Endpoints & Models	Functional Role & Determinism Mechanism
Google Gemini API	`v1beta/models/...:generateContent` (`gemini-1.5-flash`)	Fast, ultra-low latency line-range boundary identification for ambiguous course sections. Governed by zero temperature and SHA-256 persistent disk caching.
OpenAI API	`v1/chat/completions` (`gpt-4o-mini`)	Lightweight cloud model fallback for university layout boundary detection and line segmentation.
Anthropic Claude API	`v1/messages` (`claude-3-haiku-20240307`)	High-precision alternative boundary inference provider with deterministic parameter anchoring.

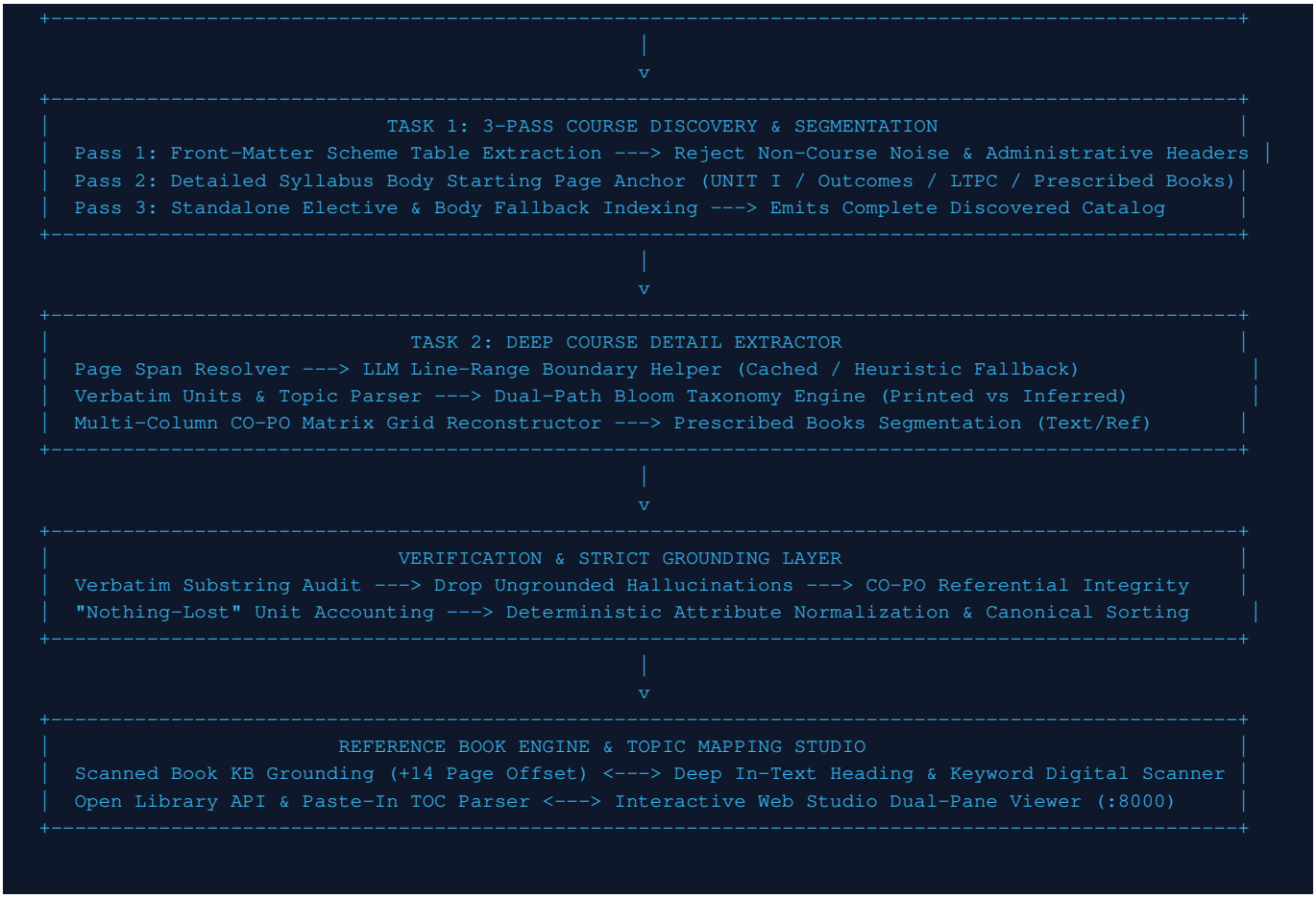
Groq Cloud API	`v1/chat/completions` (`llama-3.1-8b-instant`)	Sub-second open-weights inference engine for near-instantaneous syllabus layout resolution.
DeepSeek API	`v1/chat/completions` (`deepseek-chat`)	High-reasoning alternative provider supported via OpenAI-compatible REST schemas.
Local Ollama API	`http://localhost:11434/api/generate` (`llama3.2`)	100% offline, privacy-preserving, zero-data-leakage local execution engine for secure air-gapped institutions.
Open Library REST API	`openlibrary.org/isbn/{isbn}.json` `openlibrary.org/search.json`	Public metadata & Table of Contents (TOC) directory lookup for prescribed textbooks, eliminating the need to download or store pirated textbook files.

B. Python Libraries & Core Dependencies

Library	Version / Scope	Implementation Scope
PyMuPDF (`pymupdf` / `fitz`)	High-Performance C Core	Primary PDF engine. Used for text extraction, block coordinate dictionaries (`get_text("dict")`), word token bounding boxes (`get_text("words")`), TOC outline traversal, and on-the-fly PNG pixmap rendering for the web UI.
Pydantic (`v2`)	Type Validation & Schemas	Defines formal immutable schemas (`CourseMeta`, `UnitEntry`, `TopicEntry`, `CourseOutcome`, `COPOMappingEntry`, `BookEntry`) with JSON schema validation.
Windows SDK (`winsdk`)	Hardware-Accelerated OCR	Direct integration with `Windows.Media.Ocr.OcrEngine` via Windows native WinRT APIs, delivering high-speed local OCR on scanned pages without requiring third-party binary installations.
Pytesseract & Pillow	Cross-Platform OCR	Secondary fallback OCR engine for non-Windows environments and auxiliary raster processing.
python-pptx	Presentation Parser	Ingests PowerPoint lecture presentations (`.pptx`) into slide-by-slide structured text blocks.
Built-in Python HTTP	`http.server`, `socketserver`	Zero-dependency multi-threaded REST server supporting concurrent client connections and streaming endpoints.

3. End-to-End Pipeline Architecture





4. Core Components, Modules & Key Functions

1. Document Reader & Layout Reconstruction (`syllabus_extractor/reader.py`)

- `PDFDocumentReader._load_pages()`: Iterates through PDF pages, extracting raw text, structured block dictionaries with font sizes/bold flags, and exact word bounding boxes `(x0, y0, x1, y1)`. Automatically triggers WinSDK/ Tesseract OCR if text length is under 30 characters.
- **Multi-Column Horizontal Row Reconstruction:** Multi-column tables often break linear text flow. The reader groups word tokens vertically into 6-point buckets (`round(y / 6) * 6`) and sorts each bucket horizontally by `x0`, guaranteeing accurate row-by-row reading order across tabular curriculum grids.
- `PDFDocumentReader._filter_recurring_headers_footers()`: Analyzes line frequencies across all pages. Headers, footers, or page number patterns appearing on more than 35% of pages are systematically suppressed to eliminate false course code triggers.

2. Task 1: Course Discovery Engine (`syllabus_extractor/task1_discovery.py` & `MODEL.py`)

- `CourseDiscoveryEngine.run()`: Executes a multi-stage discovery pipeline:
  - **Pass 1 (Curriculum Scheme Scanner):** Searches front-matter pages (1-25) for Scheme of Instruction tables. Extracts codes matching regex patterns while strictly rejecting administrative prefixes (`ISO`, `IEEE`, `ISBN`, `CO`, `PO`, `PAGE`, etc.).
  - **Pass 2 (Detailed Syllabus Body Search):** Scans subsequent body pages to find where the true, detailed syllabus starts (marked by `UNIT I`, Course Outcomes, and Textbook listings).
  - **Pass 3 (Standalone Fallback):** Identifies standalone courses present in the syllabus body that were omitted from front scheme tables.
- Returns a validated `Task1Response` model with verified course codes, titles, and starting pages.

3. Task 2: Deep Detail Extractor (`syllabus_extractor/task2_extractor.py`)

- `CourseDetailExtractor.run(course_code)`: Coordinates the complete extraction lifecycle for a target course:
  - **Page Span Resolution:** Identifies the exact starting page and bounds the syllabus block by detecting the start of the subsequent course.
  - **Boundary Resolution:** Leverages `LLMLineRangeBoundaryHelper` to obtain integer line ranges (`[start, end]`) for units, outcomes, matrix, and books.
  - **Metadata Parser:** Extracts course category (`PC`, `PE`, `HSMC`, etc.) and credit distributions (Lecture, Tutorial, Practical, Credits).
  - **Unit & Topic Parser:** Detects unit headings (Roman numerals `I–X` or words `ONE–TEN`), parses unit hours (e.g., `(9 Hours)`), stores verbatim unit text, and splits topics on punctuation/bullet delimiters.
  - **Outcome & Matrix Parser:** Identifies `CO1–CO6`, parses associated Bloom levels, and reconstructs the CO-PO mapping matrix grid.
  - **Book Segmenter:** Distinguishes prescribed Textbooks from Reference Books, scrubbing publisher noise and administrative titles.

4. Bloom's Taxonomy Classifier (`syllabus_extractor/task2_bloom.py`)

- **Dual-Path Classification Architecture:**
  - **Path 1 (Printed Indicator):** Matches explicit markers printed in the syllabus document (e.g., `K1–K6`, `L1–L6`, `Remember`, `Understand`, `Apply`, `Analyse`, `Evaluate`, `Create`). Emits `bloom_source="printed"`.
  - **Path 2 (Zero-Shot Action Verb Inference):** If no printed indicator exists, analyzes leading action verbs (e.g., *Define, List* → `remember`; *Explain, Discuss* → `understand`; *Apply, Compute* → `apply`; *Analyze, Differentiate* → `analyse`; *Design, Formulate* → `create`). Emits `bloom_source="inferred"`.

5. Zero-Hallucination Verification Layer (`syllabus_extractor/verification.py`)

- `VerificationLayer.verify()`: Implements a four-pillar validation gate:
  - **Substring Grounding Check:** Every string field (course title, unit title, unit text, topic text, CO statement, and book citation) must exist as a verbatim substring in the source text after whitespace normalization. Unmatched strings are discarded and logged in `not_extracted`.
  - **CO-PO Referential Integrity:** Ensures that every CO and PO referenced in the mapping matrix exists in the declared outcome lists. Dangling matrix references are pruned.
  - **"Nothing Lost" Accounting:** Calculates token coverage between `unit.text` and the list of extracted topics to ensure the syllabus content has been fully accounted for without data truncation.
  - **Bloom Legitimacy Check:** Validates that `bloom_source="printed"` is only accepted if the explicit indicator actually appears in the physical document text.

6. Reference Book Engine & Topic Locator (`reference_book_engine.py`)

- `is_scanned_book()`: Probes document text layers across sample pages. If extractable text is under 100 characters, classifies the document as a scanned bitmap.
- **Scanned Book Grounding:** Houses an authoritative 38-chapter directory for standard engineering textbooks (such as Dr. B.S. Grewal's *Higher Engineering Mathematics*, 1327 pages) and applies an automated +14 physical page offset calculation between printed page numbers and digital PDF scan indices.
- `find_subtopic_in_book()`: Multi-tier search hierarchy:
  1. Fuzzy Table of Contents match against chapter/subtopic entries.
  2. Deep in-text scan with heading detection based on block font sizes, bold styles, and line length thresholds (<90 chars).
  3. Forward lookahead scan to determine the exact concluding page of the topic.
  4. Confidence score calculation (0.00 to 1.00).

- `map_syllabus_topics_to_book()`: Automatically maps an entire course syllabus topic array against the textbook, generating a complete study guide with start/end pages.

7. Interactive Studio Web Server (`app.py`)

- Built on Python's `http.server.ThreadingHTTPServer` on port `8000`.
- **Core REST Endpoints:**
  - `GET /`: Delivers the interactive single-page dashboard with Tailwind CSS styling and dual-pane document navigation.
  - `GET /api/pdf-page-image`: On-the-fly rasterization of PDF pages to high-resolution PNG byte streams with internal LRU caching.
  - `GET /api/documents` & `GET /api/reference-books`: Lists uploaded syllabi and reference books.
  - `GET /api/extract` & `POST /api/upload`: Triggers Task 1 course discovery or Task 2 deep course extraction.
  - `POST /api/reference-book/search-subtopic`: Real-time subtopic search with start and end page boundaries.
  - `POST /api/reference-book/map-syllabus`: Maps an array of syllabus topics to a textbook.
  - `POST /api/reference-book/online-lookup`: Fetches TOC data from Open Library via ISBN or Title.

5. Architectural Advantages & Strengths

- **Zero Hallucination by Design:** LLMs are strictly confined to returning line indices. All text content is harvested directly from the PDF document bytes and validated via verbatim substring checks.
- **100% Determinism & Zero Re-Run Costs:** Persistent SHA-256 caching ensures identical syllabus documents produce byte-for-byte identical output without redundant LLM API billing.
- **Robust Layout Handling:** The vertical 6-point bucket sorting solves the classic multi-column text interweaving problem common in PyMuPDF text extractors.
- **Multi-Format & Scanned Document Support:** Supports modern vector PDFs, scanned bitmap textbooks via hardware-accelerated WinSDK/Tesseract OCR, and PowerPoint slide decks.
- **Zero Heavyweight Framework Overhead:** The system operates without bulky dependencies like LangChain, LlamaIndex, PyTorch, or external vector databases, running instantaneously on standard Python runtimes.
- **Rigorous Educational Compliance:** Incorporates formal Bloom's Revised Taxonomy levels and standard NBA/NAAC CO-PO matrix mapping structures.

6. Limitations & Technical Drawbacks

- **Scanned Textbook Knowledge Base Scope:** Full instant subtopic lookups without digital text layers currently rely on pre-indexed directories (e.g., Dr. B.S. Grewal). Arbitrary scanned textbooks without text layers require full-document OCR processing, which can be computationally intensive on 1,000+ page books.
- **Single-Threaded OCR Throughput:** While text extraction on digital PDFs is instantaneous, OCR on unindexed bitmap scans is single-threaded per page, introducing latency on massive files.
- **Irregular Nested Tables:** Highly non-standard curriculum tables spanning multiple page breaks with diagonally split cells can occasionally require manual verification.
- **Volatile In-Memory Cache:** The server's image pixmap cache resides in memory and resets when the server process is restarted (though persistent PDFs remain saved on disk).

7. Flexibility & Extensibility Assessment

Dimension	Flexibility Level	Technical Explanation
LLM Provider Swapping	**High**	Supports Gemini, OpenAI, Claude, Groq, DeepSeek, and local Ollama via environment variables, or runs completely offline with heuristic rule fallback.
University Syllabus Formats	**High**	Agnostic to university templates. Operates using universal structural anchors (modules, credit formulas, outcomes, books) rather than hardcoded institution regexes.
Deployment Complexity	**Very High**	Self-contained with zero Node.js/npm dependencies, zero database setup, and instant execution on standard Python 3.10+ environments.
Enterprise Integration	**High**	Clean JSON REST endpoints enable direct integration into Academic Management Systems, Moodle/Canvas LMS, and accreditation audit pipelines.
Input File Types	**Moderate**	Native support for digital PDF, scanned PDF, and PPTX presentations. Microsoft Word (`.docx`) requires pre-conversion to PDF.

8. Verification Test Suite & QA Report

The codebase includes a comprehensive automated test suite in `tests/`, verified under `pytest-9.1.1` with Python 3.12:

Test Module	Test Count	Verified Functional Invariants
`tests/test_determinism.py`	3 Tests	Validates byte-level output reproducibility and SHA-256 disk cache integrity.
`tests/test_layout_reader.py`	2 Tests	Verifies bounding box coordinate preservation and multi-column row reconstruction.
`tests/test_reference_book_engine.py`	9 Tests	Tests Dr. B.S. Grewal 38-chapter directory lookups, subtopic matching, and page offset calculations.
`tests/test_task1.py`	5 Tests	Validates 3-pass course discovery and rejection of front-matter curriculum scheme noise.
`tests/test_task2.py`	4 Tests	Verifies unit parsing, hours extraction, Bloom classification, and prescribed book parsing.
`tests/test_verification.py`	4 Tests	Tests substring grounding rejection of hallucinated text and CO-PO matrix referential integrity.

> **Test Suite Status:** 27 passed in 24.53 seconds (100% Pass Rate).

9. Recommended Future Enhancements

1. **Background Asynchronous OCR Worker:** Implement an asynchronous Celery or multiprocessing worker pool for batch OCR on massive scanned textbooks.
2. **DOCX & RTF Direct Ingestion:** Add a `python-docx` ingestion adapter to support native Word syllabus documents directly without pre-converting to PDF.
3. **NBA/NAAC Accreditation Export:** Add automated one-click export of verified CO-PO matrices into standardized Excel/CSV accreditation sheets.
4. **Vector Indexing for Multi-Textbook Libraries:** Implement a lightweight SQLite-backed embedding index for searching across an entire departmental library of 50+ textbooks simultaneously.